Lab 2. Implementing HPA in Kubernetes

Overview

In this lab, you will gain hands-on experience with Kubernetes autoscaling by setting up Horizontal Pod Autoscaler (HPA) for a deployment. You will generate a load to see the effects of autoscaling in action.

Prerequisites

Kubernetes cluster with metric server installed as per Lab 1.

Exercise 2.1: Deploy Sample Application in Kubernetes

In this first exercise, we will deploy a sample application in Kubernetes. This application would be used to demonstrate the HPA autoscaling strategies that are available in Kubernetes.

1. Create **`webapp.yaml`** file with contents below. This file defines a Kubernetes deployment for your sample application:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: webapp
spec:
  replicas: 2
  selector:
    matchLabels:
      app: webapp
  template:
    metadata:
      labels:
        app: webapp
    spec:
      containers:
      - image: nginx
```

```
      name: nginx
      resources:
        limits:
          cpu: "10m"
        requests:
          cpu: "10m"
```

2. Deploy the application:

```
kubectl apply -f webapp.yaml
```

3. Verify the deployment:

```
kubectl get deployments
```

```
NAME     READY   UP-TO-DATE   AVAILABLE   AGE
webapp   2/2     2            2           89s
```

4. Set up port forwarding to access the application.

In this step, we are establishing a port forwarding rule that redirects network traffic from a specific port on your local machine to the corresponding port on the Kubernetes pod hosting the server. By doing so, you enable direct access to the server via http://localhost:8080 from your local computer. This action bridges the network gap between your local environment and the isolated Kubernetes pod, allowing you to test and interact with the deployed application as if it were running locally. **It's important to run this in a new terminal tab to keep the port forwarding active throughout your testing session.**

```
kubectl port-forward deploy/webapp 8080:80
```

```
Forwarding from 127.0.0.1:8080 -> 80
Forwarding from [::1]:8080 -> 80
```

5. Test the sample application by sending an HTTP request:

```
curl http://localhost:8080/
```

```
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
<Output Truncated>
```

Exercise 2.2: Configure & Test HPA

In this exercise, we will configure the Horizontal Pod Autoscaler in Kubernetes.

1. Create an HPA resource.

This resource will autoscale your deployment based on CPU utilization.

We are starting off by creating a Horizontal Pod Autoscaler for our Kubernetes deployment using the **`kubectl autoscale`** command:

```
kubectl autoscale deployment webapp --min=2 --max=5 --cpu-percent=20
```

```
horizontalpodautoscaler.autoscaling/webapp autoscaled
```

2. Check that HPA is correctly deployed:

```
kubectl get hpa
```

```
NAME     REFERENCE            TARGETS   MINPODS   MAXPODS   REPLICAS   AGE
webapp   Deployment/webapp    0%/20%    2         5         2          23s
```

3. Generate load using Siege.

In this step, we are using the **Siege** tool to generate traffic for our application. As CPU usage surpasses the set thresholds, the HPA responds automatically by triggering its scaling mechanism. The HPA scales up the number of pod replicas to handle the increased load. This scaling up ensures that the application maintains its performance and responsiveness despite the heightened demand.

```
siege -q -c 2 -t 1m http://localhost:8080
```

4. Monitor autoscaling.

In this step, we are actively monitoring the behavior of the Horizontal Pod Autoscaler as it responds to the increased load. By executing **`kubectl get hpa`**, you can observe the HPA's real-time response to changing CPU utilization. This command displays crucial metrics such as current CPU usage, targeted usage (set at 20% in our configuration), and the current number of replicas.

HPA operates by continually monitoring specified metrics - in this case, CPU utilization - and adjusting the number of pod replicas to meet the desired target utilization. When the load increases and CPU usage rises above the 50% threshold, the HPA responds by scaling up the number of pods.

```
kubectl get hpa -w
```

```
NAME     REFERENCE            TARGETS   MINPODS   MAXPODS   REPLICAS   AGE
webapp   Deployment/webapp    20%/20%    2         5          3          85s
```

The above output shows that HPA has scaled the **webapp** deployment to 3 replicas. This scaling action is a direct response to increased CPU utilization, which has surpassed the defined threshold of 20%. The important aspects to note here are the following:

- **`TARGETS`**: Indicates the current/average CPU utilization (20%) against the target set (20%). The match here triggered the scaling.

- **MINPODS** and **MAXPODS**: Represent the minimum (2) and maximum (5) number of pods that HPA can scale to. In this case, the number of replicas has increased but is still within the defined limits.
- **REPLICAS**: Shows the current count of replicas (3), which has increased from the original count to handle the higher load.

5. HPA clean-up, delete HPA, and the deployment:

```
kubectl delete hpa webapp
kubectl delete deploy webapp
```